

Fullstack Development

Authentication / Authorization

Authentication - **authen**

- A process of verifying user identity.
- Who is the user?
- Is the user really who he/she represents himself to be?

Authorization - author

- A process of verifying a user's access level.
- Is user **X** authorized to access resource **R**?
- Is user **X** authorized to perform operation **P**?

Note

`authn` and `author` usually go hand-in-hand:

- Protected API Access
 - `authn` : Verifies who the user is (Identity).
 - `author` : Determines what the user can access (Roles & Permissions).
- Social Login (e.g., Google)
 - `authn` : User authenticates with Google.
 - `author` : User authorizes the application to access designated resources.

Approach

Rather than talking about `authn` vs `author`, let's focus on requirements:

- How do users sign up/in with credentials?
- How do users sign up/in with social accounts?
- How do we persist users' auth states?
 - So that users don't need to sign in at every request.

Part 1: Signing up/in with credential

Situation

- User fill in username and password.
- Your app creates user entry in database.
- *How do you store password?*
 - (and also compare it?)

Part 1: Signing up/in with credential

Section 1A: How to store password

6 levels of safety

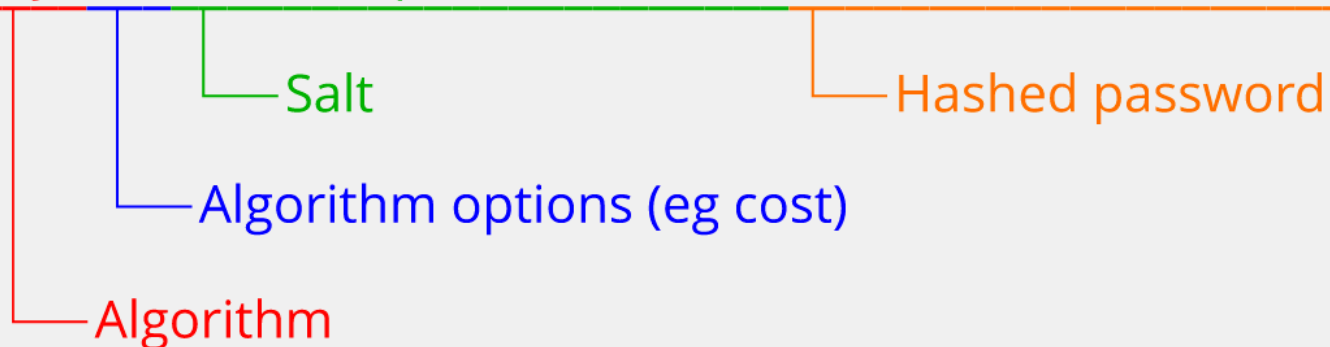
Technique	Ranking	Vulnerability
Plain text	F	All
Encryption	D	Stolen key
Hashing	C	Rainbow table attack
Salting	B	Fast computer
Salting + Cost Factor (<code>bcrypt</code>)	B+	<i>Infinity stone</i> 🏹
?	A	

Adapted from [source](#)

Note (1)

- SHA256
- Rainbow table attack
- `bcrypt` hash

`$2y$10$6z7GKa9kpDN7KC3ICW1Hi.f``d0/to7Y/x36WUKNP0IndHdkdR9Ae3K`



Note (2)

- It should be noted that the resulting "hash" contain `salt`.
 - The inclusion of `salt` is so that we do not need to keep track of it.
- But this also leave room for hacker to use it to regenerate rainbow table on the fly.
 - This is why we need cost factor.

Note (3)

- In `bcrypt`, "salt rounds" (also known as the "cost factor" or "work factor") refer to the number of iterations or rounds the hashing algorithm performs when generating a password hash.
 - Cost 10 ➔ $2^{10} = 1,024$ iterations
 - Cost 12 ➔ $2^{12} = 4,096$ iterations (4x longer than cost 10)
 - Cost 14 ➔ $2^{14} = 16,384$ iterations (16x longer than cost 10)
- Cost 10 to 12 (Industry Standard)
 - Generates a hash in roughly 100 ms to 500 ms on modern server hardware.

bcrypt example

- `git clone https://github.com/fullstack-69/auth-bcrypt.git`
- `pnpm i`
- `npx run hash`
- `npx compare`

Note on the code

- Increasing time to generate (and compare) hash with increasing `saltRounds`.
- The use of `bcrypt.compare`

Part 1: Signing up/in with credential

Section 1B: Implementation with passport

passport

- Most popular authentication middleware for `express`. ([NPM](#))
- Minimal and modular
- [500+ strategies](#)
- Confusing and [poorly documented](#) 🤪
 - [Hidden manual](#)

Let's see it

- `git clone https://github.com/fullstack-69/auth-signin-credential.git`
- `pnpm i`
- `npm run db:reset`
- `npm run dev`

Side note about the project

- MPA - HTMX
- Use `SQLite` + `drizzle`.
 - Checkout the schema.
- Try debugging in VSCode.
 - See `launch.json`.

Highlighted packages

package.json

```
{  
  "passport": "^0.7.0",  
  "passport-local": "^1.0.0"  
}
```

(Not changed for 3 years)

✦ AI Overview

The [Passport](#) npm package has not needed an update in roughly three years (since version 0.7.0 in late 2023) because its core authentication middleware is mature, feature-complete, and stable, meaning it simply does not require frequent code changes or bug fixes to function properly.  NPM

Middleware

src/index.ts

```
passport.use(  
  new LocalStrategy(  
    {  
      // Options  
    },  
    async function (email, password, done) {  
      // Verify email / password  
    },  
  ),  
);  
//  
app.use(passport.initialize());
```

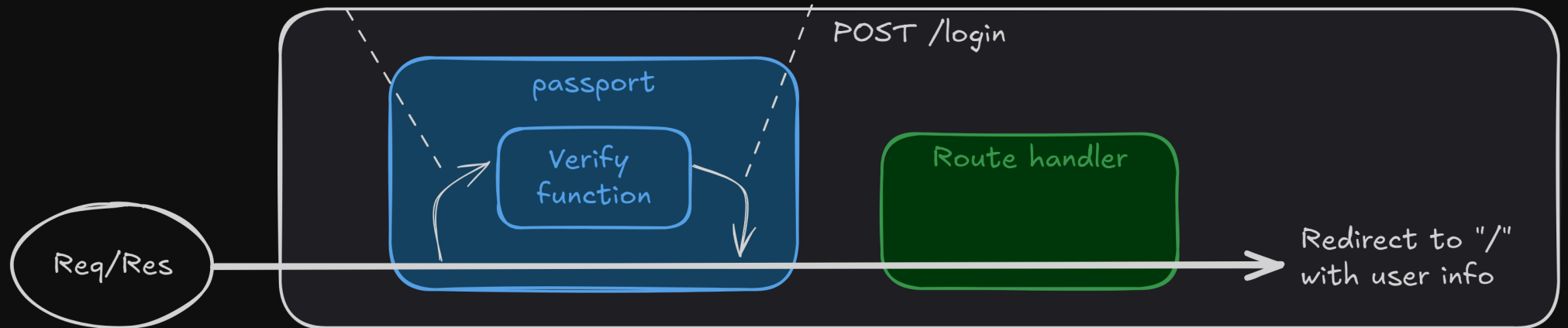
Available options

Route

```
app.post(  
  "/login",  
  passport.authenticate("local", { session: false }),  
  function (req, res) {  
    // * Passport will attach user object in the request  
  },  
);
```

- Extract email/password from "req.body" / "req.query"
- Inject credential into verify function

Return "user" object to Req



Can we do better?

Technique	Ranking	Vulnerability
Plain text	F	All
Encryption	D	Stolen key
Hashing	C	Rainbow table attack
Salting	B	Fast computer
Salting + Cost Factor (<code>bcrypt</code>)	B+	<i>Infinity stone</i>
Not storing password	A	👉👉👉

Next: Part 2